

File Access (ioutil)

Even though modern computers use high-speed SSDs, the way we program files is still based on the concept of **Linear Access**. This comes from the days of physical tapes, where you had to read from the start and move toward the end.

1. The Core Operations

Most programming languages, including Go, provide five standard ways to interact with a file:

- **Open:** Connects your code to the file so you can start working.
- **Read:** Pulls bytes out of the file into a variable (like a byte array).
- **Write:** Pushes bytes from your code into the file.
- **Close:** Tells the computer you are finished, which frees up system resources.
- **Seek:** Moves the "read head" to a specific spot in the file if you need to skip data.

2. Using the ioutil Package

For many common tasks, Go provides the `ioutil` package. It simplifies things by handling the opening and closing of files for you automatically.

Reading a File

The **`ReadFile`** function is the simplest way to get data into your program.

- **Function:** `data, err := ioutil.ReadFile("test.txt")`
- **How it works:** It opens the file, reads every single byte into a `[]byte` (byte array), and closes it immediately.
- **The "Big File" Warning:** Because this function loads the *entire* file into your RAM at once, it can crash your program if you try to read a 10GB file on a computer with only 8GB of memory. Use this only for reasonably small files.

Writing a File

The **`WriteFile`** function allows you to create a file and dump data into it in one shot.

- **Function:** `err := ioutil.WriteFile("outfile.txt", data, 0777)`
- **Arguments:**
 1. **Filename:** The name of the file to create.
 2. **Data:** The byte array or string you want to save.
 3. **Permissions:** Unix-style codes (e.g., 0777 gives everyone read/write/execute access).
- **The Limitation:** This is not for "**appending**" (adding to the end). It creates a fresh file and writes the data. If the file already existed, its old content is replaced.

Expert Takeaway:

In the professional world, `ioutil` has actually been **deprecated** since Go 1.16! The community moved these functions into the **os** and **io** packages to make things more organized.

Pro Tip: Instead of `ioutil.ReadFile`, you should now use `os.ReadFile`. It works exactly the same way but is the modern standard. When you are writing backend services, always check the file size before reading it into memory to prevent your server from running out of RAM (an "**OOM**" or Out Of Memory error).